

JavaScript

La manipulation du DOM

Table des matières

1. Présentation.....	1
2. La sélection des éléments	1
3. Modifier le contenu d'une balise	2
4. Gérer les attributs d'une balise <nomBalise attribut='valeur'>	2
5. La propriété style	3
6. La propriété classList	5
7. Comment se déplacer dans le DOM à partir d'un élément.....	5
8. Comment ajouter, modifier, supprimer des éléments dans le DOM.....	6
9. La copie d'un élément.....	7
10. Méthodes sur les balises table, tbody, thead, tfooter, tr et td	8
11. L'objet document.....	9
12. Quelques exemples.....	9

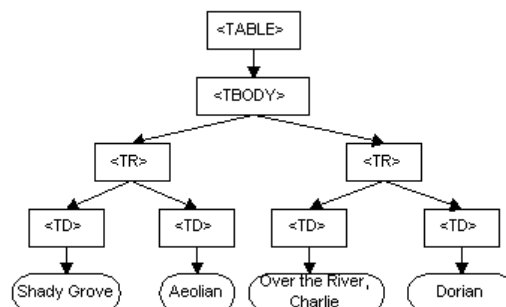
Javascript : La manipulation du DOM

1. Présentation

Le Document Object Model (ou DOM) décrit une interface permettant à des scripts d'accéder ou de mettre à jour les éléments contenus dans une structure arborescente.

Avec le Modèle Objet de Document, les programmeurs peuvent construire des documents, naviguer dans leur structure, et ajouter, modifier, ou supprimer soit des éléments soit du contenu. Tout ce qui peut être trouvé dans un document HTML ou XML peut être accédé, changé, détruit, ou ajouté en utilisant le Modèle Objet de Document. Le modèle ne contient pas uniquement la structure du document mais également son comportement et celui des objets dont il est composé.

L'objet racine correspond à l'objet document.



2. La sélection des éléments

Par la valeur de l'attribut id	<code>document.getElementById('nom')</code> Retourne la balise dont l'attribut id a la valeur 'nom'.
Par le nom de l'élément (la balise)	<code>document.getElementsByTagName('div')</code> Renvoie dans un tableau toutes les balises <div> let lesBalisesDiv = document.getElementsByTagName('div') Accès à la première balise div : let div1 = lesBalisesDiv[0]
Par la valeur de l'attribut class	<code>document.getElementsByClassName('obligatoire')</code> Renvoie dans un tableau les balises de type <.. class='obligatoire'>
Par la valeur de l'attribut name	<code>document.getElementsByName('nom')</code> Renvoie dans un tableau les balises de type <.. name='nom'>
Par un sélecteur	<code>document.querySelector('#avatar');</code> Retourne le premier élément du document qui correspond au groupe de sélecteurs passés en paramètre <code>document.querySelector('input[name=choix]:checked').value;</code> La case à cocher 'cochée' ayant un attribut name égal à 'choix'
Par un sélecteur	<code>document.querySelectorAll(".boite");</code> Retourne dans un tableau tous les éléments de classe 'boite' <code>document.querySelectorAll("select");</code> // toutes les zones de liste
Par l'objet window (adressage indirect)	<code>window['fenConsultation'].style.display = 'block';</code> accepte aussi en paramètre une variable qui contient la valeur d'un attribut id mais attention, la valeur de l'id doit commencer par une lettre let ligne = window['id' + id.value];

Les méthodes `querySelector` et `querySelectorAll` permettent l'utilisation des sélecteurs spécialisés et des pseudo-sélecteurs.

Champs dont l'id commence par 'message'	<code>document.querySelectorAll('input[id^="message"]')</code>
Champs dont l'id se termine par 'erreur'	<code>document.querySelectorAll('input[id\$="erreur"]')</code>
Champs dont l'id contient 'txt'	<code>document.querySelectorAll('input[id*="txt"]')</code>

Il est possible d'accéder directement à un élément (input en général) sans passer par la méthode `getElementById('nom')` si le script ne contient pas de variable de même nom.

<code>nom.value = 'Dupont'</code> équivalent à <code>document.getElementById('nom').value = nom</code>
--

Javascript : La manipulation du DOM

3. Modifier le contenu d'une balise

Méthodes et Propriétés	Rôle
innerHTML="texte"	Remplace le contenu de l'objet par 'texte' avec interprétation des balises html contenues dans texte
textContent="texte"	Remplace le contenu de l'objet par 'texte' sans faire la moindre interprétation du texte.
replaceChildren(element)	Remplace le contenu de l'objet par un objet du DOM (un objet Image par exemple)

Mettre à blanc toutes les balises dont l'id commence par message

```
document.querySelectorAll('[id^="message"]').forEach(element => element.innerText="");
```

4. Gérer les attributs d'une balise <nomBalise attribut='valeur'>

Notation element.methode ou element.propriete

Méthodes et Propriétés	Rôle
getAttribute('nom')	Renvoie la valeur de l'attribut 'nom' de l'objet.
setAttribute('nom', valeur)	autre possibilité : element.nom
removeAttribute(attribut)	Ajoute un attribut 'nom' avec une valeur.
	Autre possibilité : element.nom = valeur
dataset.id = valeur;	Supprime l'attribut.
attributes	Exemple : btnSupprimer.removeAttribute('disabled')
style	Modifie la valeur de l'attribut 'data-id'. Concerne tous les attributs commençant par 'data-'
onclick=	Renvoie un tableau de tous les attributs de cet élément.
	Objet donnant accès aux styles de l'élément.
	Ajoute une fonction de traitement sur l'événement click de l'élément.
	La valeur peut être le nom d'une fonction ou une fonction anonyme
value = "texte"	function() {...}
className = " "	Remplace la valeur de l'attribut value de l'objet (select par exemple)
	Modifie la valeur de l'attribut class de l'objet
	Préféré l'usage de la propriété classList (cf. 6.)

Javascript : La manipulation du DOM

5. La propriété style

Notation : `element.style.propriété`

Propriétés (exemple)	Rôle
<code>background</code>	Définir la couleur de fond de l'objet
<code>visibility="hidden visible"</code>	Masque ou affiche l'objet en le conservant dans le flux
<code>display="none block"</code>	Masque ou affiche l'objet en le supprimant flux
<code>style.color</code>	Modifie la couleur du texte
<code>fontSize</code>	Taille de la police

```
let element = document.getElementById("element");
element.style.backgroundColor = "#ff0";
element.style.color = "#0f0";
element.style.fontSize = "1.2em";
element.style.textAlign = "center";
element.style.display = 'none';
```

Il est possible de gérer les styles en utilisant les propriétés et méthodes disponibles sur l'objet `CSSStyleDeclaration` retourné par la propriété `style`.

Méthode	<code>setProperty(property, value, priority)</code>
Rôle	Définit une propriété CSS avec une valeur et une priorité spécifiées.
Exemple	<code>element.style.setProperty('color', 'red', 'important');</code> <code>element.style.setProperty('font-weight', 'bold', 'important');</code>

Méthode	<code>getPropertyValue(property):</code>
Rôle	Récupère la valeur de la propriété CSS spécifiée
Exemple	<code>const colorValue = element.style.getPropertyValue('color');</code>

Méthode	<code>removeProperty(property)</code>
Rôle	Supprime la propriété CSS spécifiée.
Exemple	<code>element.style.removeProperty('color');</code>

Propriété	<code>length</code>
Rôle	Renvoie le nombre de propriétés CSS appliquées à l'élément.
Exemple	<code>const nbStyle = element.style.length;</code>

Propriété	<code>cssText</code>
Rôle	Récupère ou définit le texte de style complet de l'élément
Exemple	<code>let allStyles = element.style.cssText;</code> <code>element.style.cssText = 'color: blue; font-size: 14px;';</code>

Remarque : L'utilisation de la propriété `style` est principalement destinée à la manipulation des styles en ligne. Si vous avez besoin de gérer des styles définis dans une feuille de style externe, vous pouvez envisager d'utiliser les méthodes liées à la feuille de style (`document.styleSheets`).

Javascript : La manipulation du DOM

6. La propriété dataset

La propriété dataset est une fonctionnalité JavaScript qui permet d'accéder aux attributs de données (data attributes) HTML personnalisés d'un élément DOM. Les attributs de données HTML personnalisés commencent généralement par data- suivis de noms personnalisés que vous pouvez définir.

```
<div id="monElement" data-id="123" data-nom="Legrand"></div>
```

Il est possible d'accéder aux valeurs de ces attributs de données personnalisés en utilisant la propriété dataset.

```
// Sélection de l'élément par son ID
var element = document.getElementById('monElement');

// Accès aux attributs de données personnalisés via la propriété dataset
var info = element.dataset.id; // "123"
var nom = element.dataset.nom; // "Legrand"
```

La propriété dataset retourne un objet DOMStringMap contenant toutes les paires clé-valeur des attributs de données personnalisés de l'élément. Les clés sont générées en supprimant le préfixe data- et en convertissant la partie suivante en camelCase. Par exemple, data-info devient info.

Pour modifier ou récupérer la valeur d'un attribut de données personnalisé :

```
// Modification de la valeur d'un attribut de données personnalisé
element.dataset.id = "24";

// Récupération de la nouvelle valeur
let nouvelID = element.dataset.id;
```

Ce système est souvent utilisé pour détecter la modification de la valeur d'un champ de saisie

Demande de modification

```
let nom = document.createElement("input");
nom.type = 'text';
nom.classList.add('form-control');
...
nom.value = element.nom;
nom.dataset.old = element.nom;
nom.style.width = '400px';
nom.onchange = () => {
    nom.value = supprimerDoubleEspace(enleverAccent(nom.value.trim().toUpperCase()));
    if (nom.value !== nom.dataset.old && verifier(nom)) {
        modifier(nom, id);
    }
};
```

La fonction modifier en cas de réussite modifie la propriété old

```
nom.dataset.old = nom.value;
```

Javascript : La manipulation du DOM

7. La propriété classList

La propriété classList permet de gérer le contenu de l'attribut class d'une balise. Techniquement elle contient chaque classe dans une collection et offre les méthodes classiques d'une collection :

Méthode	Rôle
add(nom [,nom])	Ajoute la classe CSS nom dans l'attribut class de l'élément. Exemple : <code>matricule.classList.add('erreur');</code>
remove(nom [,nom])	Supprime la classe CSS nom dans l'attribut class de l'élément.
contains(nom)	Retourne vraie si nom se trouve dans la collection.
replace(a, b)	Remplace la classe a si elle existe par la classe b.
toggle(nom)	Permute (ajoute ou retire) la classe.
item(unIndex)	Retourne la classe située à l'index unIndex dans la collection
toString()	Retourne les valeurs de la collection dans une chaîne de caractères
length	Retourne le nombre d'élément dans la collection

8. Comment se déplacer dans le DOM à partir d'un élément

Méthode et Propriété	Rôle
parentElement	Renvoie l'élément parent.
hasChildNodes()	Vérifie si un objet possède des nœuds enfants (childNodes), et Renvoie true si c'est le cas.
childNodes ou children	Renvoie un tableau de tous les nœuds à l'intérieur du nœud actuel childNodes prend en compte les nœuds texte
firstChild	Premier nœud = <code>childNodes[0]</code>
firstElementChild	Premier nœud (en ignorant le nœud texte et commentaire)
lastChild	Dernier nœud <code>childNodes[this.childNodes.length-1]</code> .
nodeName	Renvoie le nom du nœud (le nom de l'élément HTML).
nodeType	Renvoie le type du nœud : 1 élément, 2 attribut, et 3 texte.
nodeValue	Renvoie ou fixe la valeur du nœud. Cette valeur est du texte si le nœud est un nœud textuel, l'attribut lui-même si c'est un attribut, ou 'null' (valeur nulle) si c'est un élément.
nextSibling	Retourne le nœud suivant (y compris les noeud texte et commentaire)
nextElementSibling	Retourne l'élément suivant (de même niveau)
previousElementSibling	Retourne le nœud précédent (de même niveau)
parentNode	Retourne le nœud parent

Rappel : la propriété length appliquée sur un tableau retourne le nombre d'éléments du tableau.

Exemple : stocker dans un tableau les valeurs de balises 'input' placées dans des balises td
Les balises tr sont encapsulées dans la div 'lesLignes'. Elles contiennent 2 balises td.

```
for(const tr of lesLignes.querySelectorAll("tr")) {  
  let lesCellules = tr.childNodes;  
  lesTaches.push({libelle : lesCellules[1].firstChild.value, duree : lesCellules[2].firstChild.value});  
}
```

Recopier la valeur de l'input 'nom' dans l'input situé dans la troisième colonne de la balise tr dont l'id est stocké dans la variable id

```
document.getElementById(id).childNodes[2].firstChild.value = nom.value;
```

Javascript : La manipulation du DOM

9. Comment ajouter, modifier, supprimer des éléments dans le DOM

Pour ajouter un élément, il faut d'abord le créer en mémoire puis l'insérer dans le document à l'endroit souhaité.

Il existe plusieurs méthodes pour créer un élément :

Méthode	Rôle
<code>document.createElement('element')</code>	Crée un nouvel élément (une balise)
<code>document.createTextNode('chaine')</code>	Crée un nouveau nœud de texte
<code>document.cloneNode()</code>	Copie le nœud entier avec tous les nœuds enfants.

Trois méthodes permettent d'insérer un nouvel élément dans le DOM à partir d'un élément donné :

Méthode	Rôle
<code>elt.appendChild(newElt)</code>	Ajoute newElt en tant que dernier élément de elt.
<code>elt.insertBefore(enfant, suivant)</code>	Insère newElt avant l'élément suivant de elt. Si suivant n'est pas précisé enfant est ajouté comme dernier enfant
<code>elt.insertAdjacentElement('afterend', newElt)</code>	Insère newElt après elt
<code>elt.insertAdjacentElement('beforeend', newElt)</code>	Insère newElt à la fin de elt (avant sa fermeture)
<code>elt.insertAdjacentElement('afterbegin', newElt)</code>	Insère newElt au début de elt (juste après son ouverture)
<code>elt.insertAdjacentElement('beforebegin', newElt)</code>	Insère newElt avant elt

Remarque : `elt.insertAdjacentElement('afterbegin', newElt) = elt.insertBefore(newElt, elt.firstChild)`

La suppression s'effectue par la méthode suivante

Méthode	Rôle
<code>elt.removeChild(enfant)</code>	Supprime le nœud enfant dans les enfants de l'objet.

JavaScript ne permet pas de supprimer directement une balise dont on connaît la valeur de l'attribut id. Pour le faire, il faut, en partant de cette balise revenir sur le père (parentNode) et supprimer la balise à partir du père (removeChild).

```
let element = document.getElementById(id);
element.parentNode.removeChild(element);
```

Exemple : Supprimer la ligne tr contenant une cellule td contenant une balise img accessible par this :

```
let tr = this.parentNode.parentNode;
tr.parentNode.removeChild(tr);
```

Pour effacer le contenu d'une balise il faut parcourir l'ensemble des enfants et les supprimer :

```
while (unNoeud.firstChild) {
  unNoeud.removeChild(unNoeud.firstChild);
}
```

Attention : si la balise contient du texte, ce dernier ne sera pas supprimé

La solution permettant d'effacer tout le contenu : `unNoeud.innerHTML=""`

Javascript : La manipulation du DOM

Il est aussi possible de remplacer un enfant spécifique d'un élément par un autre en utilisant la méthode `replaceChild` (nouvelEnfant, ancienEnfant)

La méthode `replaceChildren(newElement)` remplace quant à elle tous les enfants d'un élément par un nouvel élément.

10. La copie d'un élément

Il est parfois nécessaire de placer plusieurs fois le même élément sur l'interface. C'est le cas par exemple lorsqu'on affiche dans un tableau des enregistrements provenant d'une source de données (table MySQL, fichier Json...) et que l'une des colonnes doit afficher la valeur sous la forme d'une zone de liste :

```
for (const element of lesEtudiants) {
  const tr = lesLignes.insertRow();
  ...
  td = tr.insertCell();
  td.style.textAlign = 'center';
  // Génération d'une balise select
  const select = document.createElement('select');
  select.classList.add("form-select");
  for (const element of lesOptions) {
    select.add(new Option(element.libelleCourt, element.id));
    select.onchange = () => modifier('idOption', select, id);
    select.value = element.idOption
  }
  td.appendChild(select);
}
```

Générer dynamiquement une balise `<select>` peut être plus coûteux en termes de performances, surtout si vous devez en générer un grand nombre. Dans ce cas, la duplication d'une balise existante peut être plus efficace car elle réduit la quantité de travail nécessaire pour créer et attacher chaque nouvelle balise.

```
// création d'une balise select
const baliseSelect = document.createElement('select');
baliseSelect.classList.add("form-select");
for (const element of lesOptions) {
  baliseSelect.add(new Option(element.libelleCourt, element.id));
}
```

```
for (const element of lesEtudiants) {
  const tr = lesLignes.insertRow();
  ...
  td = tr.insertCell();
  td.style.textAlign = 'center';
  // copie de la zone de liste de référence
  const select = baliseSelect.cloneNode(true);
  select.onchange = () => modifier('idOption', select, id);
  select.value = element.idOption
  td.appendChild(select);
}
```

Javascript : La manipulation du DOM

11. Méthodes sur les balises table, tbody, thead, tfooter, tr et td

Méthode depuis un nœud 'tbody'	Rôle
lesLignes.rows	Retourne la collection des lignes (utile pour une boucle for)
lesLignes.rows.lenght	Retourne le nombre de lignes
lesLignes.rows[n] ou .items(n)	Retourne la ligne en position n
lesLignes.deleteRow(n)	Supprime la ligne (tr) en position index
let tr = lesLignes.insertRow(index)	Insère une nouvelle ligne en position index Si la position n'est pas précisée, la ligne est ajoutée à la fin. Si l'index est supérieur au nombre de lignes une erreur se produit. Retourne une référence vers cette nouvelle ligne

Méthode depuis un nœud 'tr'	Rôle
let td = tr.insertCell(index)	Insère une cellule (td) en position index (même remarque) Retourne une référence vers cette nouvelle cellule
tr.deleteCell(index)	Supprime la cellule (td) en position index (même remarque)
tr.Cells	Retourne la collection des cellules
tr.cells.length	Retourne le nombre de cellules.
tr.cells[n] ou cells.item(n)	Retourne la cellule en position n

Exemple : génération des lignes dans le conteneur lesLignes (tbody) à partir d'une variable JavaScript 'data' contenant des objets à deux propriétés date et libelle

```
for (const element of data) {  
  // ajout d'une ligne  
  let tr = lesLignes.insertRow();  
  // ajout d'une colonne centrée contenant la date  
  let td = tr.insertCell()  
  td.style.textAlign = 'center'  
  td.innerText = element.date  
  // ajout d'une colonne contenant le libellé  
  tr.insertCell().innerText = element.libelle;  
}
```

Exemple : Alimentation d'un tableau lesElements à partir des lignes tr accessible depuis le conteneur 'lesLignes'

```
let lesElements = []  
for (const tr of lesLignes.rows) {  
  let date = tr.cells[0].innerText;  
  let message = tr.cells[1].innerText;  
  lesElement.push({date: date, message: message});  
}
```

Exemple : Afficher ou masquer des lignes contenues dans la balise d'id 'lesLignes'

```
for (const tr of lesLignes.rows)  
  if ( tr.cells[2]. firstChild.data === 'SLAM')  
    tr.style.display = "";  
  else  
    tr.style.display = 'none';
```

Javascript : La manipulation du DOM

12. L'objet document

Méthodes et Propriétés	Rôle
document.location.href	Contient l'url de la page et permet donc de changer d'url document.location.href = "relance.php?id=" + id;
document.location.search	Retourne les paramètres de l'url à partir de ? ➔ ?id=" + id;
document.write()	Permet d'écrire dans le document
document.forms[]	Tableau contenant l'ensemble des formulaires
document.img[]	Tableau contenant l'ensemble des images

13. Quelques exemples

- ➡ Générer une zone de liste

```
let id = document.createElement("select");
id.classList.add("form-control");
input.classList.add("m-2");
for (const element of lesMotifs) {
    input.add(new Option(element.libelle, element.id));
}
```

- ➡ Récupérer les paramètres passés dans l'url ex: getcourse.php?numCourse=2

```
let lesParametresUrl = new URLSearchParams(document.location.search.substring(1));
numCourse = lesParametresUrl.get("numCourse");
```

- ➡ Modifier l'attribut src si la valeur n'existe pas en la remplaçant par l'image par défaut

```
let img = new Image();
img.src = 'avatar/' + leMembre.photo;
img.onerror = function () { img.src = 'avatar/0.png' }
```

- ➡ Ne rien afficher si l'image par défaut a aussi disparu ! : On masque la div pour en afficher une autre

```
img.onerror = function () {
    if (img.src.substr(-5) === '0.png') {
        aide.style.display = 'block';
        photo.style.display = 'none';
    } else {
        img.src = '../..../avatar/0.png'
    }
};
```

Javascript : La manipulation du DOM

- ➡ Générer une balise table qui sera placée dans le conteneur 'zone' et alimentée à partir d'un tableau lesElements' composé d'objets à deux propriétés (code et libelle)

```
// création de la balise table
let table = document.createElement('table');
// définition des tableaux permettant de paramétrer le contenu de l'entête, la taille et l'alignement
let lesEntetes = ['Code', 'Libellé'];
let lesTailles = [100, 250];
let lesAlignements = ['center', 'left'];
let nbColonne = lesEntetes.length;

let tr = table.insertRow();
for(let i = 0; i < nbColonne; i++){
    let td = tr.insertCell(i);
    td.style = 'border-style:solid; text-align:' + lesAlignements[i] + '; width:' + lesTailles[i] + 'px';
    td.innerText = lesEntetes[i];
}
for (const element of lesElements) {
    tr = table.insertRow();
    let td = tr.insertCell(0);
    td.innerText = element.code;
    td = tr.insertCell(1);
    td.innerText = element.libelle;
}
document.getElementById("zone").innerHTML = "";
document.getElementById("zone").appendChild(table);
```

La classe tableau permet d'automatiser la génération de la balise table. Son constructeur prend en paramètre des tableaux permettant de paramétrer entre autres, le contenu, les styles et les classes

```
const lesColonnes = ['Code', 'Libellé'];
const lesStyles = ['text-align : center', 'text-align : left'];
const lesClasses = [];
const lesTailles = [100, 250];

const tableau = new Tableau(lesColonnes, lesTailles, lesStyles, lesClasses, 'monTableau', "");
for (const element of lesElements) {
    const lesCellules = [element.code, element.libelle];
    tableau.ajouterLigne(element, lesStyles, lesClasses);
}
// Ajouter le tableau généré au document
document.getElementById("zone").innerHTML = "";
document.getElementById("zone").appendChild(tableau.getTableau());
```

Javascript : La manipulation du DOM

- ➡ Supprimer une ligne possédant un id contenu dans une variable id

```
let ligne = document.getElementById(id);  
ligne.parentNode.removeChild(ligne);
```

- ➡ Modifier le contenu d'une cellule d'une ligne possédant un id contenu dans une variable id

```
let ligne = document.getElementById(id);  
let lesCellules = ligne.childNodes;  
lesCellules[n].innerText = valeur
```

n représente le numéro de la colonne à modifier

- ➡ Attribuer un événement change à toutes les balises 'select' de la page

```
for (const select of document.querySelectorAll('select'))  
    select.onchange = getLesElements;
```

```
let zonesDeListe = document.querySelectorAll('select');  
zonesDeListe.forEach(function(zoneDeListe) {  
    zoneDeListe.addEventListener('change', getLesElements);  
});
```

- ➡ Ajouter plusieurs classes contenues dans une variable à une élément

La propriété `classList` n'a pas de méthode directe pour définir le contenu complet à partir d'une chaîne comme `cssText` pour la propriété `style`. Il faut alors effacer les classes existantes, puis ajouter les nouvelles en utilisant la méthode `add`

La méthode `add` attend un nombre de paramètre variable, il faut donc découper la chaîne contenant les classes. Le tableau obtenu doit ensuite être décomposé en une liste d'arguments individuels, c'est le rôle de l'opérateur de décomposition `'...'`

```
var classe = "btn, btn-danger";  
var madiv = document.getElementById('madiv');  
madiv.className = "";  
madiv.classList.add(...classe.split(',').map(x => x.trim()));
```

`classe.split(',')` : permet de diviser la chaîne en un tableau de classes : `['btn', 'btn-danger']`

`map` permet de supprimer les éventuelles espaces autour de chaque classe.

`...` permet de décomposer le tableau en une liste d'arguments : `'btn', 'btn-danger'`